

SISTEMAS DISTRIBUÍDOS

ConectaShop

Relatório técnico de interoperabilidade REST e gRPC

Projeto	ConectaShop
Atividade	Interoperabilidade REST e gRPC
Participantes	Anderson; Kaio Willamis; Vinicius Durval
Escopo	Implementação e validação dos clientes REST e gRPC

Resumo

Este relatório apresenta o projeto ConectaShop, desenvolvido para a atividade de Sistemas Distribuídos sobre interoperabilidade entre aplicações. A equipe implementou clientes em Python para consumir dois serviços independentes: uma API REST para consulta de produtos e criação de cotações e um serviço gRPC para verificação de saúde e cálculo de frete. Os servidores utilizados na integração podem pertencer a outros grupos, conforme a dinâmica de sorteio da atividade; o repositório também inclui servidores locais de teste para facilitar a validação.

Os testes registrados demonstram dois cenários relevantes. Na primeira execução REST, o serviço remoto estava indisponível e as cinco chamadas falharam por recusa de conexão. Em uma execução posterior, os cinco testes REST foram aprovados, com respostas HTTP 200, 404 e 422 conforme o caso. O cliente gRPC foi estruturado para validar cinco cenários, incluindo a operação Health, três cálculos de frete e uma entrada inválida. A solução evidencia os conceitos da apostila: request/response, recursos, contratos, códigos de status, comunicação síncrona, timeout, rastreabilidade e falha parcial.

1. Introdução

Sistemas distribuídos são compostos por componentes que executam de forma independente e trocam informações por uma rede. Essa separação permite distribuir responsabilidades, mas introduz latência, indisponibilidade e incerteza sobre o resultado de uma chamada remota. A Aula 3 da disciplina aborda HTTP, REST e comunicação síncrona como formas de organizar essa interação em um nível de abstração superior ao de sockets e protocolos de transporte.

O ConectaShop foi desenvolvido nesse contexto. O projeto não pretende constituir uma loja virtual completa; seu objetivo é demonstrar, de forma observável, como um cliente pode consumir serviços remotos por meio de dois modelos de comunicação amplamente utilizados. O primeiro modelo é REST sobre HTTP, com mensagens JSON e códigos de status. O segundo é gRPC, com contrato definido em Protocol Buffers e mensagens fortemente tipadas.

2. Objetivos

O objetivo geral é realizar a atividade prática de interoperabilidade da disciplina, demonstrando o consumo de serviços REST e gRPC por clientes independentes e validando o comportamento esperado em cenários de sucesso, erro de negócio e indisponibilidade.

Como objetivos específicos, o projeto busca:

- consumir produtos e cotações por meio de uma API REST versionada;
- calcular fretes por meio de um serviço gRPC definido em Protocol Buffers;
- validar respostas por status, corpo, tipos e valores esperados;
- propagar identificadores de requisição para permitir rastreamento;
- medir a duração das chamadas e aplicar limites de espera;
- observar a diferença entre erro de conexão, erro HTTP e erro de validação gRPC;
- comparar as características arquiteturais de REST e gRPC.

3. Equipe e responsabilidades

A equipe do projeto é composta por Anderson, Kaio Williamis e Vinicius Durval. O repositório público da equipe está associado a Kaio Williamis e concentra os clientes, o contrato protobuf, os servidores de teste e o log de execução. Como não foram fornecidas informações sobre uma divisão individual mais

detalhada, este relatório registra a responsabilidade coletiva da equipe pela implementação e validação do cliente.

Componente	Responsabilidade no projeto
Cliente REST	Enviar requisições HTTP para produtos e cotações, validar respostas e registrar resultados.
Cliente gRPC	Invocar Health e CalculateShipping usando o stub gerado, validar mensagens e tratar RpcError.
Contrato protobuf	Definir o serviço ShippingService, as mensagens e os enumeradores do domínio de frete.
Servidores de teste	Simular os serviços REST e gRPC para execução local e repetível.
Integração	Consumir servidores de outros grupos quando disponibilizados pela atividade.

4. Arquitetura da solução

A arquitetura é organizada em dois fluxos independentes. No fluxo REST, o cliente ConectaShop acessa a base `http://localhost:8080/api/v1`. No fluxo gRPC, o cliente abre um canal para `localhost:50051` e utiliza o stub `ShippingServiceStub`. Os servidores podem estar no mesmo computador durante os testes locais ou em outro ambiente durante a integração entre grupos.

Fluxo	Cliente	Servidor	Transporte	Representação/contrato
REST	<code>cliente_rest/rest_client.py</code>	API de produtos e cotações	HTTP	JSON, URIs, headers e códigos HTTP
gRPC	<code>cliente_grpc/grpc_cliente.py</code>	ShippingService	HTTP/2 sobre gRPC	Protocol Buffers e métodos RPC

A comunicação é síncrona nos dois casos: o cliente envia uma chamada e aguarda a resposta antes de validar o resultado e prosseguir para o próximo teste. Essa característica simplifica o modelo de programação, mas cria acoplamento temporal. Se o servidor não estiver disponível ou não responder dentro do limite configurado, a operação não pode ser concluída naquele momento.

5. Implementação REST

5.1 Configuração e execução

O cliente REST foi implementado em Python com a biblioteca `requests`. A URL padrão é `http://localhost:8080`; o próprio programa normaliza a base para acrescentar `/api/v1` quando necessário. O código da equipe padrão é C01, mas pode ser alterado por variável de ambiente ou argumento de linha de comando.

```
# A partir da raiz do repositório
python cliente_rest/rest_client.py
```

```
# Definindo o servidor remoto e a equipe
```

```
$env:REST_BASE_URL="http://HOST:PORTA"
$env:CLIENT_TEAM="C01"
python cliente_rest/rest_client.py
```

```
# Alternativa por argumentos
python cliente_rest/rest_client.py --base-url "http://HOST:PORTA" --team C01
```

Para validação local, o repositório inclui um servidor REST de teste na porta 8080. Ele pode ser iniciado com o comando abaixo em um terminal separado:

```
python servidor_teste/server.py
```

O cliente aplica timeout de 10 segundos, envia os headers X-Client-Team e X-Request-ID e grava os resultados em logs/rest_client.log. O log também é exibido no console.

5.2 Recursos e endpoints

Método	Endpoint	Finalidade	Resposta esperada
GET	/api/v1/products/{sku}	Consultar um produto pelo SKU.	200 com dados do produto ou 404 com PRODUCT_NOT_FOUND.
POST	/api/v1/quotes	Calcular uma cotação para uma lista de itens.	200 com subtotal, desconto e total; 422 para produto inválido.

A modelagem utiliza substantivos para identificar recursos, enquanto o método HTTP expressa a intenção da operação. Essa escolha está alinhada ao princípio de interface uniforme apresentado na apostila. O versionamento /api/v1 também oferece um ponto explícito para evolução compatível da interface.

5.3 Contrato da cotação

O corpo enviado ao endpoint de cotação possui a estrutura items, que contém objetos com sku e quantity. No servidor de teste, a lista deve conter de 1 a 5 itens distintos e cada quantidade deve estar entre 1 e 10. O cálculo aplica descontos por subtotal: 0% para valores inferiores a 50.000 centavos, 5% para valores a partir de 50.000 e inferiores a 100.000 centavos, e 10% para valores a partir de 100.000 centavos.

```
POST /api/v1/quotes
Content-Type: application/json
X-Client-Team: C01
X-Request-ID: req-...

{
  "items": [
    {"sku": "KB-100", "quantity": 2},
    {"sku": "MS-200", "quantity": 1}
  ]
}

{
  "requestId": "req-...",
  "subtotalCents": 64970,
  "discountPercent": 5,
  "discountCents": 3248,
  "totalCents": 61722
}
```

6. Implementação gRPC

6.1 Contrato Protocol Buffers

O serviço gRPC é definido no pacote shipping.v1. O contrato declara o serviço ShippingService, com os métodos CalculateShipping e Health. As mensagens têm campos tipados e os valores de zona e modalidade são controlados por enumerações. Os arquivos shipping_pb2.py e shipping_pb2_grpc.py são gerados automaticamente a partir do arquivo shipping.proto e não devem ser editados manualmente.

```
service ShippingService {
  rpc CalculateShipping(ShippingRequest) returns (ShippingResponse);
  rpc Health(HealthRequest) returns (HealthResponse);
}
```

Mensagem	Campos principais
ShippingRequest	request_id: string; weight_grams: int32; zone: ShippingZone; mode: ShippingMode.
ShippingResponse	request_id: string; price_cents: int64; estimated_days: int32; server_team: string.
HealthRequest	Mensagem vazia.
HealthResponse	status: string; server_team: string.

6.2 Execução e regras de negócio

O cliente gRPC utiliza por padrão o destino localhost:50051 e o código de equipe C01. Cada chamada usa metadata x-client-team e timeout de 5 segundos. Para a validação local, o servidor de teste pode ser iniciado em um terminal separado com o comando abaixo:

```
python servidor_teste_grpc/server.py
```

O servidor de teste rejeita pesos inferiores a 1 grama ou superiores a 30.000 gramas. Também rejeita zona ou modalidade não especificadas. O preço é calculado a partir de uma tarifa base, de um adicional por quilograma cobrado e da zona e modalidade escolhidas. O prazo é definido por uma tabela de dias estimados.

Método	Entrada	Saída ou erro
Health	HealthRequest vazio e metadata da equipe.	HealthResponse com status SERVING.
CalculateShipping	Peso, zona, modalidade e request_id.	ShippingResponse com preço, prazo e server_team.
CalculateShipping inválido	Peso igual a zero.	INVALID_ARGUMENT com detalhe INVALID_WEIGHT.

7. Procedimento de testes

Os clientes foram estruturados como suítes de testes automatizados. Cada cenário gera um request_id, executa uma chamada, mede sua duração e compara a resposta recebida com valores esperados. O

resultado é marcado como PASS ou FAIL. Essa estratégia transforma o contrato da API em verificações executáveis.

7.1 Casos REST

Teste	Requisição	Critério de aprovação
R1	GET /products/KB-100	Status 200 e unitPriceCents igual a 25.990.
R2	GET /products/XX-999	Status 404 e code PRODUCT_NOT_FOUND.
R3	POST /quotes com KB-100 x2 e MS-200 x1	Subtotal 64.970; desconto 5%; total 61.722 centavos.
R4	POST /quotes com MN-400 x1	Subtotal 119.990; desconto 10%; total 107.991 centavos.
R5	POST /quotes com XX-999 x1	Status 422 e code INVALID_PRODUCT.

7.2 Casos gRPC

Teste	Entrada	Critério de aprovação
G1	Health	status igual a SERVING.
G2	1.500 g, LOCAL, STANDARD	1.800 centavos e 2 dias.
G3	2.500 g, REGIONAL, EXPRESS	4.600 centavos e 2 dias.
G4	1.000 g, NATIONAL, STANDARD	3.400 centavos e 7 dias.
G5	Peso igual a 0	INVALID_ARGUMENT e detalhe INVALID_WEIGHT.

8. Resultados observados

8.1 Indisponibilidade do serviço REST

Na primeira execução registrada, as cinco chamadas REST falharam porque a porta 8080 recusou a conexão. O cliente registrou CONNECTION_ERROR e não recebeu status HTTP nem corpo JSON. Os tempos ficaram entre 4.023 ms e 4.094 ms. Esse cenário representa uma falha parcial: o processo cliente estava ativo, mas sua operação dependia de um serviço remoto que não estava disponível.

8.2 Integração REST aprovada

Em uma segunda execução, os cinco cenários foram aprovados. Os tempos registrados ficaram entre 2.033 ms e 2.054 ms. Os resultados confirmam que a API utilizou diferentes códigos para diferentes categorias de resultado, em vez de retornar HTTP 200 para todos os casos.

Teste	Status	Resultado observado	Duração
R1	200	Produto KB-100 encontrado;	2043 ms

		unitPriceCents=25990.	
R2	404	Produto inexistente; code=PRODUCT_NOT_FOUND.	2049 ms
R3	200	Subtotal=64970; desconto=5%; total=61722.	2054 ms
R4	200	Subtotal=119990; desconto=10%; total=107991.	2033 ms
R5	422	Produto inválido; code=INVALID_PRODUCT.	2033 ms

O log contém, para algumas falhas de conexão, uma linha preliminar com operation=UNKNOWN e outra linha posterior com o nome do caso, como R1_GET_PRODUCT. Isso ocorre porque a função de tratamento de exceção registra o erro antes que a função de teste registre o resultado final. É uma particularidade de observabilidade que pode ser aprimorada, mas não altera a conclusão sobre a indisponibilidade observada.

8.3 Resultados gRPC

O código do cliente gRPC está preparado para validar cinco cenários: Health, três cálculos de frete e uma entrada inválida. O repositório também contém um servidor gRPC de teste com as regras necessárias para produzir as respostas esperadas. Como o material fornecido não inclui uma saída de execução gRPC equivalente ao log REST, os valores G1 a G5 são apresentados neste relatório como critérios de teste e comportamento esperado, e não como evidência independente de execução registrada.

9. Análise conforme a apostila

9.1 HTTP, REST e representação

No fluxo REST, HTTP fornece a estrutura request/response. O método, a URI, os headers e o corpo formam a requisição. O status, os headers e o corpo formam a resposta. JSON é utilizado como representação dos recursos, mas não deve ser confundido com REST. O valor arquitetural está na identificação de recursos, na interface uniforme e na semântica dos métodos.

9.2 Contratos e códigos de status

O projeto utiliza códigos coerentes com a natureza do resultado. HTTP 200 indica sucesso nas consultas e cotações; HTTP 404 identifica um produto inexistente; HTTP 422 representa uma entrada semanticamente inválida, como um SKU desconhecido para cotação. Essa separação permite que o consumidor tome decisões sem depender apenas de mensagens textuais.

9.3 Comunicação síncrona, timeout e falhas

As chamadas são síncronas porque cada teste aguarda o retorno do servidor antes de decidir se passou ou falhou. O cliente REST usa timeout de 10 segundos e o cliente gRPC usa timeout de 5 segundos. O registro de CONNECTION_ERROR demonstra que uma interface padronizada não elimina a possibilidade de falha de rede ou indisponibilidade do servidor.

9.4 Statelessness e idempotência

Os clientes não dependem de uma sessão conversacional armazenada no servidor para interpretar cada chamada. O contexto necessário é enviado por headers, metadata e pelo corpo da requisição. Essa

característica é compatível com statelessness e favorece a distribuição das chamadas entre instâncias. As consultas GET são naturalmente adequadas a repetição. Já POST/PUT deve ser analisado com cuidado antes de retries automáticos: embora o servidor de teste apenas calcule uma cotação, uma operação de criação ou cobrança poderia produzir efeitos duplicados em outro domínio.

9.5 REST e gRPC em comparação

Critério	REST	gRPC
Contrato	Endpoints, métodos, JSON, headers e status HTTP.	Arquivo .proto com métodos, mensagens e tipos.
Interoperabilidade	Ampla, inclusive com clientes HTTP genéricos.	Alta entre tecnologias com suporte a protobuf e gRPC.
Erros	Códigos HTTP e corpo JSON.	StatusCode e detalhes da exceção RpcError.
Observabilidade	request ID em header e logs estruturados.	request ID na mensagem e metadata da equipe.
Adequação	Integrações públicas e APIs orientadas a recursos.	Chamadas internas tipadas e contratos mais rígidos.

10. Limitações e melhorias

A principal limitação documental é a ausência de uma saída de execução gRPC anexada ao log. O código e o servidor de teste permitem descrever o comportamento, mas os resultados G1 a G5 devem ser confirmados em uma execução registrada para que o relatório possa afirmar que todos foram aprovados.

Também seria possível aprimorar o projeto com as seguintes medidas:

- registrar uma única linha final por teste quando ocorrer uma falha de conexão, evitando a duplicidade UNKNOWN e operação nomeada;
- incluir o corpo completo das respostas nos logs, com cuidado para não registrar dados sensíveis;
- adicionar testes para headers ausentes, JSON inválido, quantidade fora do intervalo e SKUs duplicados;
- comparar tempos de resposta REST e gRPC em condições equivalentes;
- documentar explicitamente a instalação das dependências requests e grpcio;
- avaliar políticas de retry apenas para operações com semântica segura ou com chave de idempotência.

11. Conclusão

O ConectaShop atende ao objetivo da atividade ao demonstrar dois modelos de interoperabilidade entre aplicações distribuídas. No cliente REST, a equipe utilizou HTTP, JSON, recursos, headers, códigos de status, timeout e logs estruturados. No cliente gRPC, utilizou um contrato Protocol Buffers versionado, mensagens tipadas, enumeradores, metadata e códigos de erro próprios do framework.

Os resultados REST comprovam tanto a integração bem-sucedida quanto a ocorrência de uma falha distribuída. A mesma API respondeu corretamente a consultas válidas, produto inexistente, cotações com descontos e produto inválido. Quando o servidor estava desligado, o cliente identificou a recusa de

conexão, evidenciando o acoplamento temporal da comunicação síncrona. Essa combinação de sucesso e falha é particularmente importante para a disciplina, pois mostra que REST e gRPC organizam a comunicação, mas não eliminam os problemas fundamentais da rede.

Assim, o projeto fornece uma base prática para os próximos temas da disciplina, especialmente timeout, retry, idempotência, tolerância a falhas, contratos e evolução de serviços.